

1. Need and Benefits of Component Lifecycle

What is Component Lifecycle?

- The Component Lifecycle is the series of events that happen from the moment a React component is created to the moment it is removed from the DOM
- Every React component goes through these phases automatically
- Lifecycle methods give developers control over what happens at each stage

Why Do We Need Lifecycle Methods?

- To initialize data when a component is created
- To fetch data from APIs when a component appears on screen
- To respond to prop or state changes
- To clean up resources like event listeners and timers before a component is removed
- To optimize performance by controlling when components re-render
- To integrate with third-party libraries that need access to the DOM

Benefits of Component Lifecycle

- Predictable execution: Methods run in a specific, reliable order every time
 - Better user experience: Show loading spinners, fetch data, and update UI smoothly
 - Performance optimization: Prevent unnecessary re-renders
 - Resource management: Clean up subscriptions, timers, and event listeners to prevent memory leaks
 - Easier debugging: Log component behavior at specific moments
 - Third-party integration: Initialize libraries like charts or maps when the DOM is ready
-

2. Lifecycle Hook Methods (Phases)

Overview of All Lifecycle Methods

React lifecycle methods are divided into three main phases:

Phase	Purpose
Mounting	Component is created and inserted into the DOM
Updating	Component updates when props or state change
Unmounting	Component is removed from the DOM

Phase 1: Mounting (Birth)

Called when a component is being created and inserted into the DOM.

Method	When It Runs	Purpose
constructor()	First, before rendering	Initialize state and bind methods
static getDerivedStateFromProps ()	Before rendering	Update state based on props
render()	After constructor	Return JSX to display

componentDidMount()	After component is inserted	Fetch data, set up subscriptions, timers
---------------------	-----------------------------	--

Phase 2: Updating (Growth)

Called when a component's props or state change.

Method	When It Runs	Purpose
static getDerivedStateFromProps() ()	Before rendering	Update state based on props
shouldComponentUpdate()	Before rendering	Decide if re-render is needed (performance)
render()	After shouldComponentUpdate	Return updated JSX
getSnapshotBeforeUpdate()	Just before DOM updates	Capture information (e.g., scroll position)
componentDidUpdate()	After component updates	Perform side effects based on changes

Phase 3: Unmounting (Death)

Called when a component is being removed from the DOM.

Method	When It Runs	Purpose
<code>componentWillUnmount()</code>	Just before removal	Clean up subscriptions, timers, event listeners

Phase 4: Error Handling (React 16+)

Called when there is an error during rendering.

Method	When It Runs	Purpose
<code>static getDerivedStateFromError()</code>	When an error occurs	Update state to show fallback UI
<code>componentDidCatch()</code>	After an error occurs	Log error information

3. Sequence of Steps in Rendering a Component

Mounting Sequence (First Time)

1. `constructor()` is called
2. `static getDerivedStateFromProps()` is called
3. `render()` is called
4. React updates the real DOM
5. `componentDidMount()` is called

Updating Sequence (When State or Props Change)

1. static `getDerivedStateFromProps()` is called
2. `shouldComponentUpdate()` is called
3. If `shouldComponentUpdate` returns false, the update stops here
4. If `shouldComponentUpdate` returns true, continues to step 5
5. `render()` is called
6. `getSnapshotBeforeUpdate()` is called
7. React updates the real DOM
8. `componentDidUpdate()` is called

Unmounting Sequence (When Component is Removed)

1. `componentWillUnmount()` is called
 2. Component is removed from the DOM
-

4. Detailed Explanation of Each Method

Mounting Phase Methods

`constructor(props)`

- Called first when a component is created
- Must call `super(props)` to access `this.props`
- Used to initialize state and bind methods
- This is the only place to assign `this.state` directly

Example:

text

```
constructor(props) {  
  super(props);  
  this.state = {  
    name: props.initialName || 'Guest',  
    count: 0  
  };  
  this.handleClick = this.handleClick.bind(this);  
}
```

```
}
```

`static getDerivedStateFromProps(props, state)`

- Called before every render (both mounting and updating)
- Rarely used in modern React
- Returns an object to update state, or null if no update is needed
- Useful when state depends on props changes

Example:

```
text
```

```
static getDerivedStateFromProps(props, state) {  
  if (props.userId !== state.prevUserId) {  
    return {  
      prevUserId: props.userId,  
      userData: null  
    };  
  }  
  return null;  
}
```

`render()`

- The only required method in a class component
- Must return JSX, React elements, or null
- Should be a pure function (no side effects)
- Cannot modify state directly inside render()

Example:

```
text
```

```
render() {  
  return (  
    <div>  
      <h1>Hello, {this.state.name}!</h1>  
      <p>Count: {this.state.count}</p>  
      <button onClick={this.handleClick}>Click Me</button>  
    </div>  
  );  
}
```

componentDidMount()

- Called after the component is inserted into the DOM
- Perfect for making API calls
- Used to set up subscriptions or timers
- Can access DOM elements directly
- Good place to initialize third-party libraries

Example:

text

```
componentDidMount() {  
  fetch('https://api.example.com/users')  
    .then(response => response.json())  
    .then(data => this.setState({ users: data }));  
  
  this.interval = setInterval(() => {  
    this.setState({ time: Date.now() });  
  }, 1000);  
}
```

Updating Phase Methods

static getDerivedStateFromProps(props, state)

- Same as mounting phase
- Runs before every render when props change

shouldComponentUpdate(nextProps, nextState)

- Called before rendering when state or props change
- Returns true (re-render) or false (skip re-render)
- Used for performance optimization
- Prevents unnecessary re-renders

Example:

```
text
```

```
shouldComponentUpdate(nextProps, nextState) {  
  if (this.state.count === nextState.count) {  
    return false; // No re-render needed  
  }  
  return true; // Re-render needed  
}
```

render()

- Same as mounting phase
- Returns updated JSX

getSnapshotBeforeUpdate(prevProps, prevState)

- Called just before the DOM updates
- Captures information from the DOM (e.g., scroll position)
- The value returned is passed to `componentDidUpdate`

Example:

```
text
```

```
getSnapshotBeforeUpdate(prevProps, prevState) {  
  if (prevState.messages.length < this.state.messages.length) {  
    return this.listRef.scrollHeight - this.listRef.scrollTop;  
  }  
  return null;  
}
```

componentDidUpdate(prevProps, prevState, snapshot)

- Called after the component updates
- Used to perform side effects based on changes
- Can access the snapshot value from `getSnapshotBeforeUpdate`

Example:

```
text
```

```
componentDidUpdate(prevProps, prevState, snapshot) {
```



```
if (snapshot !== null) {
  this.listRef.scrollTop = this.listRef.scrollHeight - snapshot;
}
if (this.props.userId !== prevProps.userId) {
  this.fetchUserData(this.props.userId);
}
}
```

Unmounting Phase Methods

`componentWillUnmount()`

- Called just before the component is removed from the DOM
- Used to clean up resources
- Cancel timers, intervals, and subscriptions
- Remove event listeners

Example:

```
text
componentWillUnmount() {
  clearInterval(this.interval);
  document.removeEventListener('click', this.handleDocumentClick);
}
```

Error Handling Phase Methods

`static getDerivedStateFromError(error)`

- Called when an error occurs during rendering
- Updates state to display a fallback UI
- Does not have side effects

Example:

```
text
```

```
static getDerivedStateFromError(error) {  
  return { hasError: true, error: error };  
}
```

`componentDidCatch(error, info)`

- Called after an error occurs
- Used to log error information
- Can send error reports to a service

Example:

```
text
```

```
componentDidCatch(error, info) {  
  console.log('Error:', error);  
  console.log('Component Stack:', info.componentStack);  
  logErrorToService(error, info);  
}
```

5. Lifecycle Methods in Functional Components

With React Hooks, functional components can also use lifecycle features:

Class Component Method

Functional Component Hook

`componentDidMount` +

`useEffect()`

`componentDidUpdate` +

`componentWillUnmount`

`componentDidMount`

`useEffect(() => {}, [])`

componentDidUpdate	useEffect(() => {})
componentWillUnmount	useEffect(() => { return () => {} }, [])
shouldComponentUpdate	React.memo()

6. Summary Table of All Methods

Phase	Method	Called	Purpose
Mounting	constructor()	Before render	Initialize state, bind methods
Mounting	getDerivedStateFromProps()	Before render	Sync state with props
Mounting	render()	During render	Return JSX
Mounting	componentDidMount()	After render	API calls, subscriptions
Updating	getDerivedStateFromProps()	Before render	Sync state with props
Updating	shouldComponentUpdate()	Before render	Performance optimization
Updating	render()	During render	Return updated JSX

Updating	<code>getSnapshotBeforeUpdate()</code>	Before DOM update	Capture DOM information
Updating	<code>componentDidUpdate()</code>	After render	Side effects after update
Unmounting	<code>componentWillUnmount()</code>	Before removal	Clean up resources
Error	<code>getDerivedStateFromError()</code>	When error occurs	Show fallback UI
Error	<code>componentDidCatch()</code>	When error occurs	Log error information

7. Common Use Cases

`componentDidMount` Use Cases:

- Fetching data from an API
- Setting up event listeners
- Starting timers or intervals
- Initializing third-party libraries
- Accessing DOM elements

`shouldComponentUpdate` Use Cases:

- Preventing unnecessary re-renders
- Performance optimization for large lists
- Comparing props and state changes

`componentDidUpdate` Use Cases:

- Performing actions after state changes
- Fetching new data based on prop changes
- Updating DOM based on new state
- Animations based on updates

componentWillUnmount Use Cases:

- Clearing timers and intervals
- Removing event listeners
- Canceling API requests
- Cleaning up subscriptions
- Preventing memory leaks